02-614 Week 11
String Algorithms

Aidan Jan

November 12, 2026

Text Editors

How do text editors store strings? This is an interesting question since when a file is opened in a text editor, it is assumed that the text contained in the file will change... a lot. Previously, we were given a string which we assumed would be constant. However, that is no longer the case, and many of the data structures already covered would not work since we don't have the ability to do preprocessing.

Data Structures

Line-Oriented Data Structures

The file is represented as a linked list with each node containing a linked list.

- This is efficient in inserting lines, since it just means creating a new node.
- However, it is suboptimal for inserting characters since we would have to iterate through the text.
- In the worst case, if the text file is a single, really long line, then this is just equivalent to the naïve way of storing a string.

Gap Buffer

Imagine T as an array, and create a buffer of some size after the cursor.

- If we move the cursor, then copy the character after the buffer to the front. The buffer does not change size with this operation.
- If we type, we begin to fill the buffer.
- The downside is if we fill up the buffer, we must copy the entire string to a new, larger array in order to create a new buffer. Guessing an optimal buffer size becomes important.

This is the strategy Emacs uses.

Piece Tables

Let T be a series of text that is fixed (assumed to not change). Let M be an array (also text) that can grow. Now, we represent the string as a table.

T/M	P	length
T	0	10
M	0	5
T	11	5
M	6	32

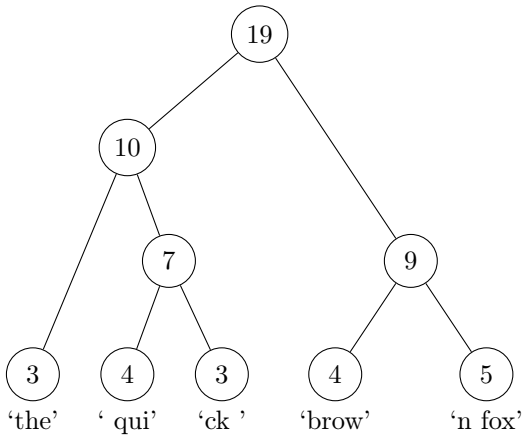
To get a string, we read off of the table. In this example, our string would be composed of characters $T[0:10] + M[0:5] + T[11:16] + M[6:38]$.

- The benefit of this approach is that once we find the correct position in the table, inserting or deleting text happens in constant time.
  - To insert, we split a row into two, the first one the section in T we want to edit, and the second in M, containing our edit.
  - To delete, split the row in T into two rows in T, and just change the starting indices to slice the string.
- When the file is saved, all the changes are written to T.
- The length of the table is never longer than the length of the string. However, M can grow in an unbounded manner if edits are large.

This is the basis of the strategy Code OSS (VS Code) uses. More specifically, Code OSS uses their own designed "piece trees".

Ropes

Consider the the string "the quick brown fox". The Ropes strategy breaks it into a series of nodes (not necessarily separated by word boundaries). One possible example may be [(the), (qui), (ck), (brow), (n fox)]. Now, we make these nodes the leaf nodes of a binary tree.



For the tree, we would use some balanced search tree (Splay, AVL, Red/Black). Nodes store the number of characters represented.

- To reconstruct the string, we read the data in an in-order traversal of the tree.
- To insert text, split the leaf to create a new node, and rebalance the tree.
- To delete text, edit the text in the nodes directly.

Grammars

A grammar is a set of rewriting rules called "productions".

$$\alpha \to \beta$$

where α and β are both strings. Additionally, an alphabet is defined as $\Sigma \cup \mathcal{N}$, where $\mathcal{N}$ are non-terminal characters.

For example:

$$S = abNac$$
$$N \to ccN$$
$$N \to \epsilon$$

Here, N is a non-terminal since N can be substituted with other characters. All other (lowercase) characters here are terminals because they cannot be substituted. In this case, we may define:

$$S' = abccNac$$
$$S'' = abccccNac$$

We get this by substituting in the N with the characters it can represent. The **language** of a grammar is the set of all strings with no non-terminals that that can be generated by the productions.

- $abccaac$ is NOT a valid production, since there is an extra a that cannot be produced.
- ϵ in languages represent empty string.

Chomsky Hierarchy

There are four types of grammars:

1. Regular grammars (Regex)
2. Context-free grammar
3. Context-sensitive
4. Unrestricted

These are a hierarchy because each grammar is a subset of its subsequent grammar. (i.e., A context-free grammar is also a context-sensitive grammar, but not the other way around.)

Regex can be checked fast (using dynamic programming), context-free grammars can be checked in polynomial time, context-sensitive grammars are NP-hard to check, and unrestricted grammars are undecidable.

Regular Grammars

Regular grammars have productions only of these forms:

- $W \to aU$
- $W \to a$
- $W \to \epsilon$

Regular grammars have the property that their strings can only grow from the end.

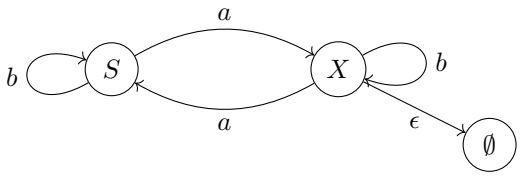
Example

Consider the following regular grammar:

$$S \to aX|bS$$
$$X \to aS|bX|\epsilon$$

The | symbol just means OR.

It is hard to see what this grammar represents, but we can visualize regular grammars as a state graph (automata).



Each non-terminal represent a state, and productions are represented as state transitions. From this, it is much clearer what the grammar represents, which is a string containing an odd number of a's.

Regular Expressions

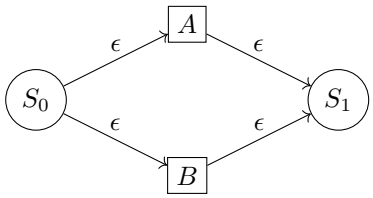
A regular expression is a string over the alphabet $\Sigma \cup \{|, (,),\ ^*\}$ such that

- a for $a \in \Sigma$ is a regular expression (match a)
- (A) if A is regular expression (match A)
- $A|B$ if A, B are regular expressions (match if either A or B matches)
- A^* if A is a regular expression (matches if $S = 0$ or more repetitions of something that matches A)
- AB if A, B are regular expressions (matches if A matches prefix, B matches the rest)

A regular expression produces matching strings. (i.e., a regular language cna be specified by a regular grammar).

All of these rules can be represented by DFAs.

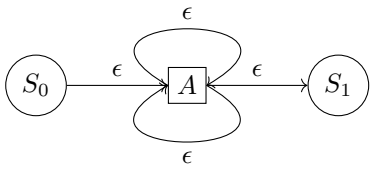
- The frst two cases are trivial, using the basic state diagram described above.
- For the $A|B$ case, do the following:



  where the A and B nodes are blackboxes (DFAs for A and B, respectively.). We just connect the starting state to the starting states of A and B using epsilon edges, and their respective ending states to our ending state.

- For the AB case, we connect $S_0 \to A \to B \to S_1$ in series with epsilon edges.

- For the A^* case, do a loop connecting the start and end nodes of A with epsilon edges, allowing for A to execute zero or multiple times.



Note that any regular expression can be represented by a regular grammar (DFA), and vice versa.

- As a side note, the regular expression DFAs containing the ϵ edges are actually non-deterministic finite automata (NFAs). However, all NFAs can be represented as DFAs.
- NFA $\equiv$ DFA $\equiv$ Regular grammar $\equiv$ Regular expression

Context-Free Grammars

These are defined as

$$W \to \beta$$

where W is a non-stem in $\mathcal{N}$, and β is any string in $\Sigma \cup \mathcal{N}$.

For example:

- $S \to ()|(Q)$
- $Q \to SQ|S$

Note that here, we can grow out of the middle instead of just the end, like regular grammars. Instead of representing this as a DFA, it is represented as a tree.

Chomsky Normal Form

Chomsky Normal Form is a grammar such that all productions have the form

- $W \to UV$
- $W \to a$

As it turns out, every context-free grammar can be represented in Chomsky Normal form. For example,

- $S \to W_1 W_2$
- $W_1 \to abQ$
- $W_2 \to cxZ$